

Chapter 10. Gate 매핑과 가중치 설계 방법론

이 챕터에서 배우는 것

- 9장의 매핑표는 "결과"다 — 그 결과에 이르는 "과정"을 되짚어 방법론으로 뽑아낸다
- "실패 모드에서 출발한다, Gate에서 출발하지 않는다"는 원칙이 실제로 무엇을 뜻하는지
- Gate 가중치를 어떤 기준으로 정할지, Book-forge의 기본값 (`gate_a_tcr_weight=0.4` 등)이 왜 그 숫자인지
- 완벽한 가중치를 처음부터 노리지 않는 점진적 조정이 왜 더 안전한지

이런 분이 먼저 읽으면 좋습니다: 9장을 읽고 "그런데 왜 ChapterDrafterAgent는 Gate C·D·E를 쓰고 Gate B·F·G는 안 쓰는가"를 스스로 도출하고 싶어진 분. 자신의 프로젝트에 Agent-Evaluator를 붙이려는데 "어떤 Gate부터, 어떤 가중치로 시작해야 할지" 막막한 분.

10.1 문제 제기 — 9장의 표는 결과이지 과정이 아니다

9장 (§ 9.2)은 Book-forge의 14개 에이전트가 7개 Gate 중 정확히 무엇을 채우는지 표로 정리했다. 그 표는 정확하지만, "왜 그렇게 나눴는가"는 답하지 않는다. 왜

`ChapterDrafterAgent` 는 `SLAConfig` 와 `ThreatSeverityConfig` 를 쓰고

`AgentRoleConfig` 는 안 쓰는가? 왜 Gate A의 가중 평균은

`gate_a_tcr_weight=0.4` (9장 § 9.4)인가, `0.7` 이면 안 되는가? 이 질문들에

"Book-forge 코드가 그렇게 돼 있으니까"라고만 답하면 순환 논리다. 이 챕터는 그 도출 과정 자체를 방법론으로 분리해, 여러분 자신의 프로젝트에도 적용할 수 있게 만든다.

10.2 실패 모드에서 출발한다, Gate에서 출발하지 않는다

Agent-Evaluator SDK의 실전 이식 가이드는 이 원칙을 단정적으로 말한다. **"Gate에서 출발하면 피상적인 매핑이 나온다."** `required_keywords=[]` 나

`alignment_threshold=0.7` 같은 값을 근거 없이 고르게 되고, 판정이 실제 품질과 무관해진다. 대신 실패 모드에서 출발하면, 하나의 실패가 여러 Gate에 동시에 걸쳐 있다는 사실도 자연스럽게 드러난다.

이 방법론은 세 가지 질문으로 요약된다.

1. **망했다는 것을 어떻게 아는가?** (주로 Gate A/C — 목표를 놓쳤거나 근거 없는 내용을 만들어냈거나)
2. **잘못된 방향으로 가고 있다는 것을 어떻게 아는가?** (주로 Gate B/F/G — 반복에 빠졌거나, 역할을 벗어났거나, 판단 근거를 설명 못 하거나)
3. **겉으로는 잘 돌아가지만 내부에서 문제가 생기는 경우는?** (주로 Gate D/E — 느려지거나, 위험한 콘텐츠가 섞이거나)

이 세 질문을 `ChapterDrafterAgent` (7~8장에서 이미 코드를 본 에이전트)에 실제로 적용해보자. 이미 아는 결론(9장의 표)을 거꾸로 재현하는 것이지만, **과정 자체를 손으로 밟아보는 것이** 이 절의 목적이다.

질문	ChapterDrafterAgent에 적용하면	도출되는 Gate
망했다는 것을 어떻게 아는가?	챕터가 목차가 요구한 내용을 실제로 담았는가	Gate A — <code>ground_truth</code> 와의 TCR·Accuracy 블렌딩(9장 § 9.4)
내부적으로만 문제가 생기는 경우는?	겉보기엔 그럴듯한데 RAG 소스에 없는 내용을 지어냈는가	Gate C — <code>rag_mode=True</code> 가 자동 켜는 <code>HallucinationDetector</code> (7장 § 7.4)
내부적으로만 문제가 생기는 경우는?	로컬 모델이 응답을 너무 오래 끌어 저자를 기다리게 하는가	Gate D — <code>SLAConfig(p95_ms=60_000, p99_ms=90_000)</code>
잘못된 방향으로 가고 있다는 것을 어떻게 아는가?	소스가 외부 PDF/문서라 프롬프트 인젝션이 섞여 들어올 수 있는가	Gate E — <code>ThreatSeverityConfig()</code> (8장)

거꾸로, **이 표에 없는 Gate가 왜 없는지**도 같은 질문으로 설명된다.

`ChapterDrafterAgent` 는 한 번 호출되고 끝나므로 "반복"이라는 사건 자체가 없다 (Gate B 무관). 다른 에이전트와 직접 상호작용하지도 않는다 (Gate F 무관, 이건 ReviewPanel의 몫 — 5장). 이 결론은 9장 (§ 9.3)이 이미 말한 "N/A는 결함이 아니라 정직함"과 정확히 같은 곳에 도달한다. 다만 이번에는 그 결론이 "SDK가 그렇게 나왔다"가 아니라 **"이 세 질문에 답이 없기 때문"**이라는 근거를 갖는다.

 **개발자 TIP:** 팀 프로젝트라면 과거 장애·버그 목록이 실패 모드 카탈로그의 상당 부분을 이미 채워준다. 1인 프로젝트라면 "아직 안 일어났지만 걱정되는 것"의 목록으로 시작해도 된다. 3장이 정리한 Book-forge의 다섯 실패 유형(무응답·환각·드리프트·반복·경로 침범)도 그 자체로 훌륭한 출발점이다.

10.3 가중치를 정하는 기준 — 회복 난이도·발생 빈도·사용자 가시성

Gate를 무엇으로 채울지 정했다면, 다음 질문은 "이 Gate가 실패했을 때 실제로 얼마나 심각한가"다. Agent-Evaluator 실전 이식 가이드는 세 기준으로 이 무게를 정하라고 권한다. **회복 난이도**(되돌리기 어려운가), **발생 빈도**(자주 일어나는가), **사용자 가시성**(사용자가 그 실패를 직접 보는가).

Book-forge의 실제 기본값을 이 기준으로 다시 읽어보면 왜 그 숫자인지 설명이 된다.

 **출처:** `Book-forge/CLAUDE.md` 에 문서화된 공식(실제 소스 코드 발췌가 아니라 개발자가 문서로 정리해둔 수식)

```
_a_score = gate_a_tcr_weight × TCR컴포넌트 + (1 - gate_a_tcr_weight) × 나머지 평균
```

`gate_a_tcr_weight=0.4` 는 "완료했는가"(TCR)에 40%, "얼마나 잘 완료했는가"(Accuracy 등)에 60%를 준다는 뜻이다. Book-forge의 핵심 가치는 "챕터 초안이 존재하는 것"이 아니라 "그 초안이 목차·소스를 실제로 반영한 것"이다(00_기획안.md). 완료 자체는 사용자 가시성이 낮다(파일이 생겼다는 것만으로는 저자가 만족하지 않는다) — 그래서 완료율보다 품질 쪽에 더 무게를 준 것이 이 기본값의 근거다. 반대로

`gate_b_loop_weight=0.0` (CLAUDE.md에 문서화된 기본값)은 발생 빈도 기준으로 설명된다. Book-forge의 14개 에이전트 중 반복 개념이 있는 것은 `revise()` (6장) 하나뿐이다. 나머지 13개 태스크에서는 항상 N/A인 신호에 기본 가중치를 크게 주면, 대부분의 채점에서 의미 없는 잡음만 늘어난다.

Agent-Evaluator 실전 이식 가이드가 소개하는 다른 프로젝트(Lecture_forge) 사례와 비교하면 이 판단 기준이 프로젝트마다 다르게 적용된다는 것이 더 분명해진다. 그 프로젝트는 Gate E(보안)에 `3.0`이라는 가장 높은 가중치를 줬다 — "보안 사고는 회복이 가장 어렵다"는 이유였다. Book-forge도 같은 이유로 보안 트래커를 상시 켜두지만 (`enable_security_metrics=True`, CLAUDE.md), 책 저술 파이프라인이라는 맥락상 외부 공개 API를 직접 노출하지 않으므로 Gate E에 별도의 높은 가중치를 주지는 않는다. **같은 원칙(회복 난이도가 높으면 무게를 더 준다)이 프로젝트마다 다른 숫자로 이어지는 것이 바로 이 방법론의 요점이다.** 정답 숫자를 외우는 게 아니라, 이 세 기준으로 여러분의 프로젝트를 다시 채점하는 것이 핵심이다.

10.4 점진적 롤아웃 — 완벽한 가중치를 처음부터 노리지 않는다

Agent-Evaluator 실전 이식 가이드는 여기서 분명한 경고를 남긴다. **"처음부터 높은 가중치를 매기면 배포 블로킹이 너무 잦아져, 팀이 평가 도입 자체를 회피하게 됩니다."** 권장 순서는 모든 가중치를 `1.0` (또는 SDK 기본값)으로 시작해 몇 차례 실제로 돌려본 뒤, **실제로 문제를 일으킨 Gate만** 올리는 것이다.

이 정신은 Book-forge 저장소 자체의 이력에서도 확인된다.

`use_korean_tokenizer=True` (16장 § 16.4에서 자세히 다룬다)는 처음부터 켜져 있던 값이 아니다. 한국어 산출물에서 Gate A·C의 정확도 채점이 조사 · 어미 때문에 실제보다 낮게 나오는 것을 실측으로 확인한 뒤에야 추가된 옵트인 파라미터다. **완벽한 설정을 미리 예측해 한 번에 다 켜는 대신, 실제로 필요가 확인될 때마다 하나씩 조정해온 것**이 Book-forge가 실제로 걸어온 경로다. 이 챕터의 방법론(실패 모드 도출 → 가중치 설계)과 16장의 실제 배선(`.env` 오버라이드) 사이의 관계는 정확히 이렇다. **이 챕터는 "무엇을, 왜"를 답하고, 16장은 "그것을 실제 코드로 어떻게 조정 가능하게 만들었는가"를 답한다.**

직접 해보기

여러분의 에이전트(또는 3장 "직접 해보기"에서 이미 점검한 에이전트) 하나를 골라, § 10.2의 세 질문을 순서대로 적용해보라. "망했다는 걸 어떻게 아는가?"에 답이 나오면 Gate A/C 후보, "잘못된 방향으로 가고 있다는 걸 어떻게 아는가?"에 답이 나오면 Gate B/F/G 후보, "내부에서만 문제가 생기는 경우는?"에 답이 나오면 Gate D/E 후보다. 답이 안 나오는 질문은 그 Gate가 필요 없다는 뜻이지, 빠뜨린 것이 아니다. 그다음 § 10.3의 세 기준(회복 난이도·발생 빈도·사용자 가시성)으로 방금 고른 Gate들의 우선순위를 매겨보라. 처음부터 정확한 숫자를 정하려 하지 말고, § 10.4처럼 "가장 걱정되는 것 하나"에만 높은 가중치를 주는 데서 시작하면 된다.

이 챕터의 핵심

- **Gate에서 출발하면 피상적인 매핑이 나온다.** "이게 망하면 어떻게 아는가/잘못된 방향인가/내부 문제인가" 세 질문에서 거꾸로 Gate를 도출해야 근거가 생긴다.
- **하나의 실패 모드가 여러 Gate에 동시에 걸릴 수 있다.** 1:1 매핑을 강제할 필요는 없다.
- **가중치는 회복 난이도·발생 빈도·사용자 가시성 세 기준으로 정한다.**
`gate_a_tcr_weight=0.4`, `gate_b_loop_weight=0.0` 이라는 Book-forge의 기본값도 이 기준에서 나온 결론이다.
- **완벽한 가중치를 처음부터 노리지 않는다.** 기본값에서 시작해, 실제로 문제를 일으킨 Gate만 조정하는 편이 안전하다 — 이 배선은 16장이 이어받는다.

참고 자료

- `src/book_forge/eval/monitor.py` — `build_book_monitor()` 의 Gate 가중치 기본값
- `Book-forge/CLAUDE.md` — Gate 가중치 커스터마이징 절
- `Agent-Evaluator/Media/Book/Part_VI_실전이식가이드/Chapter_24_Gate_매핑_전략.md` — 이 챕터가 재구성한 원본 방법론(기존 프로젝트 이식용으로 쓰였지만, 신규 설계에도 그대로 적용된다)

다음 챕터는 이렇게 도출한 Gate 선택이 아직 다루지 않는 영역 — LLM을 아예 호출하지 않는 순수 정적 검증기 3종을 다룬다.